

Real-Time MIDI Call-and-Response Generation Using Autoregressive Transformers

Wang Zitong, Hu Sitong

August 2026

Abstract

This paper tests whether an offline Anticipatory Music Transformer (AMT) can support real-time MIDI call-and-response without retraining. The primary contribution is an external adaptation architecture combining adaptive MIDI-VAD, phrase control, asynchronous playback, and speculative scheduling. A direct 100-call endpoint replay, a harmonized 27,000-row candidate comparison, a 63,000-row module ablation, an 18,000-row scheduler replay, and a blind study provide separate evidence. Under a fully specified structural-compliance composite, controlled AMT improves over shared-batch raw AMT from 0.560968 to 0.732610; style compliance accounts for much of the late-stage gain, while non-style structure rises from 0.558971 to 0.676704. Adaptive MIDI-VAD attains F1=0.712 at the stated two-second deadline, similar to fixed 800 ms (0.708) but with fewer premature commits (38 versus 68). A 150 ms pre-commit overlap reduces mean scheduler latency from 161.303 to 91.721 ms and underruns from 20.744% to 5.244%. Among 41 retained listeners, A4 is not rated conclusively above raw AMT, and the motif baseline is preferred to A6. The evidence supports an engineering adaptation and measurable structural control, not perceptual superiority or universally accurate turn segmentation.

Keywords: symbolic music generation; MIDI; call-and-response; Transformer; temporal point process; real-time interaction; objective evaluation; blind listening; ablation study; latency logging; speculative preload

1 Introduction

Call-and-response is a foundational pattern in live improvisation, but realizing it computationally requires symbolic generators that can react on a performance clock rather than only complete offline continuations.

Autoregressive Transformers capture rich symbolic-music structure in offline generation, yet their stepwise decoding is poorly matched to live co-performance. Direct deployment creates four linked bottlenecks: synchronization with human timing, reliable phrase-end detection, latency spikes during inference, and structural degradation in high-entropy continuations. This paper therefore asks whether these limits can be reduced through external architecture and control layers without retraining the underlying model.

The **primary contribution** is the external system method: adaptive turn detection, phrase control, and speculative scheduling make a frozen offline AMT operable in a live MIDI loop. The stopping-time formulation is its turn-taking basis, while the endpoint benchmark, harmonized structural evaluation, module ablation, latency replay, and blind study are supporting tests with deliberately separate claim boundaries.

2 Related Work

This section reviews the four areas that shape the proposed system: symbolic music representations, generative sequence models, real-time co-performance systems, and evaluation of generative music.

2.1 Symbolic Music Representation

Early symbolic music generation often used piano-rolls or fixed time-step grids, but rigid quantization can create sparse matrices and discard micro-timing such as rubato and swing [1]. Event-based representations address this limitation through MIDI-like streams with `Note-On`, `Note-Off`, and relative `Time-Shift` events [13, 6]. The Anticipatory Music Transformer further models symbolic music with temporal point-process events and asynchronous controls [16]. However, representation-level advances alone do not specify how continuous MIDI input should be segmented into interactive turns, especially when chords, hesitation pauses, and rubato-like timing occur in live performance.

2.2 Sequence Models for Music Generation

Symbolic music generation has moved from memoryless or recurrent models such as LSTMs [4] to self-attention-based Transformer architectures [17, 6]. AMT extends this line toward controllable temporal-event generation [16], but it remains primarily an offline model. Live co-performance adds endpoint detection, turn scheduling, and latency stability as deployment constraints. These models therefore do not directly address phrase-end detection or scheduler-level latency, which become necessary when offline symbolic generation is embedded into live interaction.

2.3 Real-Time Human-AI Co-Performance

Real-time human-AI co-performance systems must handle input capture, turn-taking, latency, and synchronization in addition to generation quality. MIDI-performance research emphasizes timing stability in interactive systems [12]; Scramble Live illustrates the combination of learned models and live control in Max/MSP-style environments [14]; and recent work couples Max/MSP front ends with Python generative back ends [8]. These systems show the importance of live control surfaces, but they leave open how an offline autoregressive symbolic model can be wrapped with endpoint logic, buffering, and fallback behavior without retraining. This study addresses that deployment gap through virtual MIDI routing and multithreaded asynchronous scheduling.

2.4 Evaluation of Generative Music

While human listening tests remain vital for evaluating musical quality, they are expensive and difficult to scale. Prior surveys emphasize that objective metrics for symbolic and generative music are useful for reproducible structural analysis, but they cannot fully replace perceptual or human-centered evaluation [7, 10]. For real-time call-and-response, evaluation must also separate structural response quality from runtime behavior such as endpoint-to-first-MIDI latency and buffer underrun. The present evaluation therefore treats objective metrics, ablation evidence, scheduler logs, and a fixed-stimulus blind listening study as complementary rather than interchangeable forms of evidence.

3 Temporal Point Process Modeling and Autoregressive Generation

3.1 Continuous-Time MIDI Event Representation

The performance is represented as a continuous-time event sequence

$$E = \{e_i = (t_i, d_i, p_i)\}_{i=1}^N, \quad 0 \leq t_1 \leq \dots \leq t_N, \quad (1)$$

where t_i is onset time, $d_i > 0$ is duration, and $p_i \in \{0, \dots, 127\}$ is MIDI pitch. Non-strict onset ordering permits chords; Section 4.2 clusters near-simultaneous Note-On events before estimating onset density.

3.2 Conditional Autoregressive Factorization

For a human Call C and response $R = \{e_j^r\}_{j=1}^n$, the frozen AMT decoder uses

$$P(R | C) = \prod_{j=1}^n P(e_j^r | C, e_{<j}^r), \quad (2)$$

with event-level factorization

$$P(e_j^r | \cdot) = P(t_j^r | \cdot)P(d_j^r | t_j^r, \cdot)P(p_j^r | t_j^r, d_j^r, \cdot). \quad (3)$$

The causal model supplies continuation probabilities; endpoint detection, scheduling, and phrase controls remain external and do not retrain its parameters.

3.3 Dynamic Turn Segmentation Based on Continuous-Time Stopping Time

Let T_{last} be the most recent Note-On and $\tau = t - T_{\text{last}}$ the current silence. Under a point-process model with intensity $\lambda(t)$, the no-arrival probability is [2]

$$S(\tau) = \exp \left[- \int_0^\tau \lambda(T_{\text{last}} + u) du \right]. \quad (4)$$

For the short online window, the implementation uses a local constant-rate approximation. If $u_1, \dots, u_K$ are the most recent clustered onset times,

$$\mu_{\text{tempo}} = \max \left(0.25, \frac{K-1}{u_K - u_1} \right), \quad S(\tau) = \exp(-\mu_{\text{tempo}}\tau). \quad (5)$$

With confidence boundary $\theta = 0.05$, the candidate cutoff and stopping condition are

$$\tau_{\text{cutoff}} = \frac{\ln(1/\theta)}{\mu_{\text{tempo}}}, \quad (6)$$

$$t - T_{\text{last}} > \tau_{\text{cutoff}}. \quad (7)$$

Thus the silence allowance shortens as recent onset density rises. This is an adaptive timer, not a proof of musical phrase completion; chord correction, revocable confirmation, and the direct endpoint benchmark in Section 5.8 define and test its operational behavior.

4 Real-Time MIDI Call-and-Response System Design

4.1 Cross-Environment Architecture Based on Virtual MIDI

The system uses a modular Virtual MIDI Bus architecture rather than a monolithic offline-generation pipeline. Physical MIDI input, host audio rendering, neural inference, and MIDI output scheduling are decoupled so that equivalent routing chains can be used across operating systems and digital audio workstations.

In Windows, the routing chain can use loopMIDI with REAPER, Decent Sampler, or another host; in macOS, it can use the IAC Bus with Logic Pro or an equivalent DAW. The system is therefore defined by the interaction chain itself: physical MIDI input, virtual MIDI routing, a Python inference engine, and host-based rendering.

Figure 1 separates MIDI capture and host rendering from the Python back end, which maintains the Call buffer, detects endpoints, decodes asynchronously, applies phrase controls, schedules playback, and writes diagnostic logs.

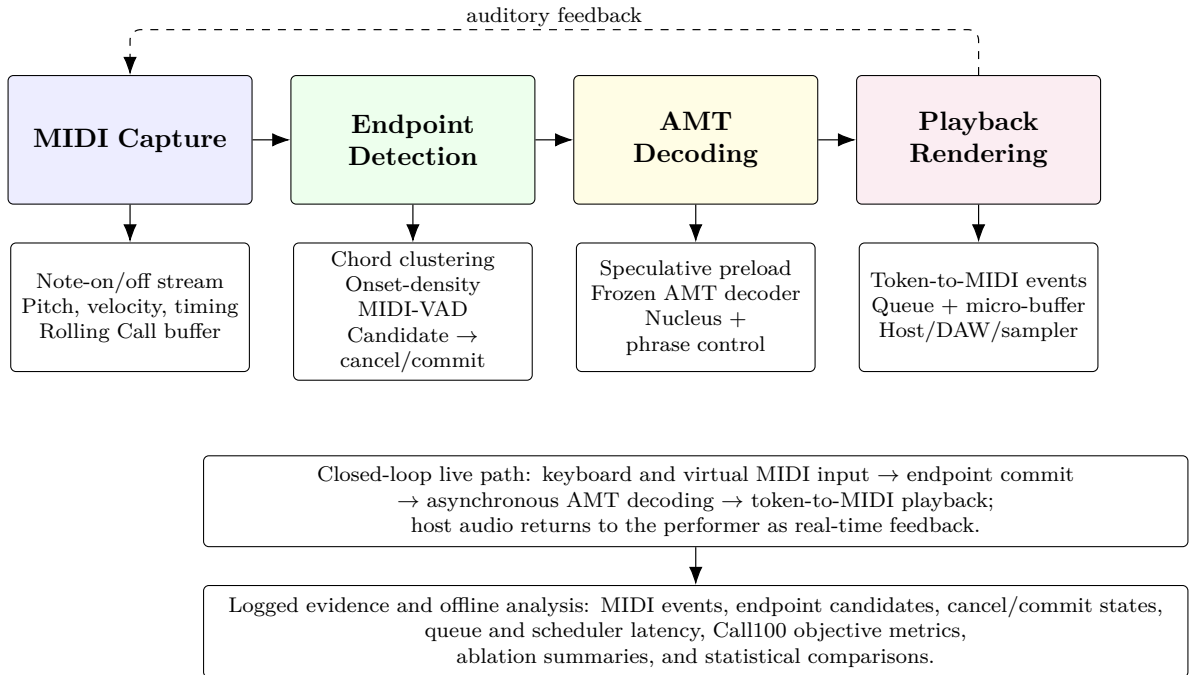


Figure 1: System overview of the real-time MIDI call-and-response framework. The figure shows the four online stages, the auditory feedback loop, and the logged evidence used for offline evaluation.

The online path is therefore capture $\rightarrow$ candidate/commit $\rightarrow$ AMT decoding $\rightarrow$ MIDI playback. Speculative work may begin at a candidate, but only committed Calls enter the audible path; canceled work is discarded.

4.2 Engineering Implementation of the Intelligent MIDI Endpoint Detection Module

The MIDI-VAD daemon polls Note-On events, updates the clustered onset window, and evaluates Equations (6)–(7). Its state machine is

$$\text{Listening} \rightarrow \text{Candidate} \rightarrow \begin{cases} \text{Cancel} \rightarrow \text{Listening}, \\ \text{Commit} \rightarrow \text{Inference}. \end{cases} \quad (8)$$

where a new Note-On cancels an unconfirmed candidate; commit freezes the Call for inference.

4.2.1 Chord-Clustering Correction for Onset-Density Estimation

Counting every note in a chord would inflate μ_{tempo} . Note-On events within

$$\Delta t_{\text{cluster}} \approx 0.08 \text{ s.} \quad (9)$$

are merged into one onset cluster. The value is an engineering setting, not a learned constant. Online accompaniment work explicitly allows imperfect performance detection [3]; related MIDI voice separation groups onsets within 35 ms and separately uses an 80 ms busy-note allowance after onset-group time [11]. Neither value transfers directly to this detector. Here 80 ms corrects density only, and Section 5.8 tests 0, 40, 80, and 120 ms sensitivity rather than treating it as universal.

4.2.2 Two-Stage Revocable Candidate Endpoint and Confirmation Window

When Equation (7) is first satisfied, the system records $T_{\text{candidate}}$ and waits

$$\Delta T_{\text{confirm}} \approx 0.15 \text{ s.} \quad (10)$$

A Note-On within this interval cancels the candidate; otherwise it commits. The 150 ms value is not a perceptual threshold. Synchronous music can demand roughly 20–30 ms latency [9], while tolerance varies strongly with task and feedback topology [15]. For this turn-taking system, 150 ms is a debounce/latency compromise tested against 0, 75, and 250 ms in Section 5.8; preload can overlap, but not extend beyond, this same interval.

4.3 Asynchronous Streaming Decoding and Micro-Buffered Scheduling

An inference thread places playable events in a thread-safe queue while a playback thread converts them to MIDI. Playback begins after an 80 ms micro-buffer. For generated event duration $D(y_k)$ and per-event inference time $\tau_{\text{infer}}(k)$, an underrun is avoided while

$$\Delta T_{\text{buffer}} + \sum_{k=1}^j D(y_k) > \sum_{k=1}^{j+1} \tau_{\text{infer}}(k), \quad \forall j \in \{1, 2, \dots, n-1\}. \quad (11)$$

This overlap changes scheduling, not intrinsic AMT inference time.

4.3.1 Speculative Preload for Candidate Endpoints

At candidate time, the engine snapshots the Call and may start decoding while MIDI monitoring continues. A commit reuses available tokens; a cancel discards them. The scheduler model used in Section 5.9 is

$$L_{\text{first}} = \max(0, \tau_{\text{first}} - \Delta T_{\text{confirm}}) + \Delta T_{\text{buffer}}. \quad (12)$$

Here τ_{first} is time to the first playable token. Because $\Delta T_{\text{preload}} = \Delta T_{\text{confirm}} = 150 \text{ ms}$, no 250 ms post-confirmation horizon exists: preload occupies only the revocable candidate interval. Canceled candidates trade temporary computation for lower committed-path delay.

4.4 Relative Interval Representation and Nucleus Sampling Control in a High-Entropy Setting

The decoder receives relative timing and pitch movement,

$$\Delta t_i = t_i - t_{i-1}, \quad \Delta p_i = p_i - p_{i-1}, \quad (13)$$

and applies temperature-scaled nucleus sampling [5]. For sorted token probabilities $P_T(x_{(i)})$, it retains the smallest prefix satisfying

$$K = \min \left\{ k \mid \sum_{i=1}^k P_T(x_{(i)}) \geq p \right\}. \quad (14)$$

where p is preset-dependent (typically 0.95) and

$$P_T(x_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}. \quad (15)$$

These controls reshape sampling but impose no tonal rule; tonal and metrical projection, when enabled, occurs later in the explicit style layer.

4.5 Phrase-level Music Control Layer

The phrase controller derives global limits from the Call, rejects repetition failures during sampling, matches response duration, and supplies a deterministic fallback only when no playable neural output exists. It remains external to AMT parameters.

4.5.1 CallProfile-based Phrase Intention Analysis and ResponsePlan Construction

After commit, the CallProfile is

$$P_{\text{call}} = [T_{\text{call}}, N_{\text{call}}, \Omega_{\text{pitch}}, \bar{p}_{\text{call}}, H_{\text{pitch}}, \nabla_{\text{melody}}, \rho_{\text{rhythm}}, T_{\text{repeat}}]^T. \quad (16)$$

The terms encode duration, note count, pitch range and mean, pitch-class distribution, first-order contour, onset density, and repeated-tail status. They produce

$$\text{Plan}_{\text{resp}} = \{T_{\text{target}}, N_{\text{target}}, \Omega_{\text{allow}}, CPR_{\text{limit}}, Share_{\text{max}}, K_{\text{resample}}\}. \quad (17)$$

which fixes target duration/note count, allowed register, continuous-repetition and dominant-pitch limits, and the resampling budget.

4.5.2 Repetition-tail Compression and Repetition Suppression

A repeated Call tail (at least three same-pitch events or dominance in the final window) is compressed to at most two prompt events. During decoding, a token is rejected if it would exceed either CPR_{limit} consecutive equal pitches or dominant-pitch share $Share_{\text{max}}$. Rejection is retried up to K_{resample} times; exhaustion invokes a nearby consonant event-level repair around $\bar{p}_{\text{call}}$. This repair is distinct from the whole-phrase motif fallback below.

4.5.3 Motif-transformation Fallback and Duration Matching

(1) Whole-phrase fallback trigger and mapping. The motif generator is called only when decoding returns zero playable events and fallback is enabled. Rejected individual events instead use the event-level repair described above; they do not replace the phrase. For an empty response, the live system transforms the cleaned Call tail:

$$R_{\text{fallback}} = T_{\text{scale}} \circ T_{\text{clip}} \circ T_{\text{shift}} \circ T_{\text{inverse}}(C_{\text{tail}}). \quad (18)$$

Here C_{tail} is the last $\min(N_{\text{call}}, N_{\text{target}})$ notes after prompt cleaning. The live fallback preserves their chronological order and maps each $e_i = (t_i, d_i, p_i)$ to

$$\hat{t}_i = T_{\text{end}}(C) + \lambda(t_i - t_{\text{tail},0}), \quad \hat{d}_i = \text{clip}(\lambda d_i, d_{\text{min}}, d_{\text{max}}), \quad (19)$$

where $T_{\text{end}}(C)$ is the end time of the Call, $t_{\text{tail},0}$ is the first onset in the selected tail, and

$$\lambda = \max \left(0.2, \frac{T_{\text{target}}}{\max(T_{\text{tail}}, \epsilon)} \right). \quad (20)$$

Pitch is generated by inversion around the Call mean pitch and a small contour shift:

$$\hat{p}_i = \text{clip} \left(2\bar{p}_{\text{call}} - p_i + \delta, p_{\text{min}}^{\text{allow}}, p_{\text{max}}^{\text{allow}} \right). \quad (21)$$

For live fallback, $\delta = -5$ semitones for descending Calls and $+5$ otherwise. Durations are clipped to configured minima/maxima; rejected fallback events pass through the same repetition repair before acceptance. The offline motif baseline, also used by the A4 branch if it is ever activated, selects the tail under the fixed experiment seed, uses $\delta \in \{-5, -3, 2, 3, 5, 7\}$, applies the same pitch-share safeguards, re-sorts by onset, and finally applies the chosen style projection. Section 5.7 reports activation separately from event repairs.

(2) Response duration matching. Before rendering, duration matching computes

$$T_{\text{raw}} = \max_i (t_i^r + d_i^r) - \min_i t_i^r. \quad (22)$$

$$\gamma = \frac{T_{\text{target}}}{T_{\text{raw}}}. \quad (23)$$

If $T_{\text{raw}} \in [0.80T_{\text{target}}, 1.25T_{\text{target}}]$, only duration bounds are enforced. Longer responses use bounded proportional scaling. Shorter responses mix normalized original onset position q_i with sequence position $i/(n-1)$:

$$q'_i = (1 - \beta)q_i + \beta \frac{i}{n-1}, \quad \beta = \text{clip} \left(0.35 + 0.45 \left(1 - \frac{S_{\text{raw}}}{S_{\text{target}}} \right), 0.35, 0.80 \right), \quad (24)$$

$$d'_i = \text{clip} \left(d_i \min \left(2, \frac{T_{\text{target}}}{T_{\text{raw}}} \right), d_{\text{min}}, d_{\text{max}} \right). \quad (25)$$

Here S denotes onset span. Raw duration, adjusted duration, and stretch factor are logged for ablation analysis.

4.6 Chromatic Free Improvisation and Optional Style Constraint Layer

Chromatic mode leaves pitch class and rhythmic grid unconstrained. The optional `pentatonic_2bar_4_4` post-processor instead makes a specific style contract explicit; it does not change AMT probabilities.

4.6.1 Chromatic Mode

This default retains nucleus sampling and phrase safety controls but applies no hard scale, cadence, meter, or quantization rule.

4.6.2 Engineering Implementation of the `pentatonic_2bar_4_4` Style Constraint Layer

The constrained mode applies five deterministic stages: (i) truncate to eight beats in 4/4; (ii) project pitch to

$$P_{\text{pent}} = \{p \in \mathbb{Z} : (p \bmod 12) \in M_{\text{mode}}\}, \quad p'_i = \Pi_{P_{\text{pent}}}(p_i); \quad (26)$$

(iii) quantize onset and duration to the selected grid,

$$t'_i = \Delta t_{\text{grid}} \text{round}(t_i / \Delta t_{\text{grid}}); \quad (27)$$

(iv) move unstable notes on beats one and three toward stable degrees; and (v) correct the final cadence and fold or replace any interval exceeding Δp_{max} . These same targeted rules motivate separating style compliance from non-style structure in the evaluator.

5 Experimental Design, Objective Evaluation, and Blind Listening

The evaluation follows five claim-specific layers: endpoint segmentation, candidate comparison, module attribution, scheduler replay, and fixed-stimulus listening. No layer is treated as a substitute for another.

5.1 Experiment Hierarchy and Evaluation Goals

The evaluation is organized to avoid a single-score interpretation of the system. Because the proposed method combines musical control, module-level engineering, and runtime scheduling, each claim is tested at the level where it can be observed most directly. Figure 2 summarizes this structure before the individual datasets, metrics, and results are introduced.

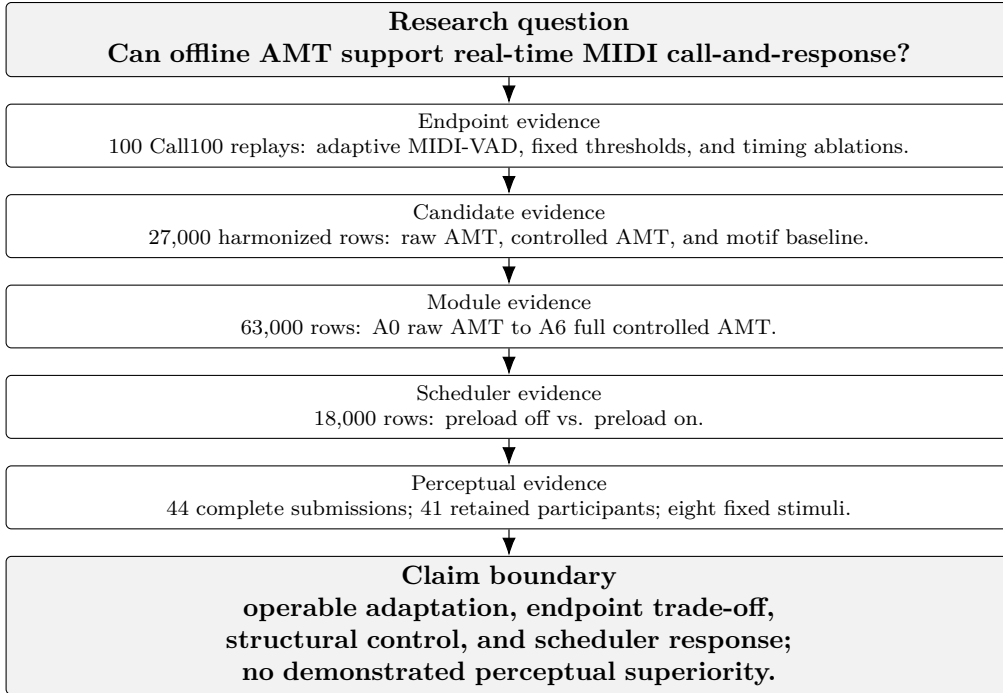


Figure 2: Experiment hierarchy and evidence chain.

Endpoint replay tests turn segmentation directly; the remaining layers test structural control, module attribution, scheduler behavior, and listener preference in that order. The primary system claim therefore does not depend on interpreting the structural composite as perceived musical quality.

5.2 Construction of the Call100 Benchmark

Call100 contains 100 call phrases and extends early demonstrations into a mixed-distribution robustness test. It is built in two stages. First, it reuses Call50 without modifying its files: 10 artificial stress-test calls and 40 real public MIDI fragments. The artificial calls deliberately probe likely failure modes, including clear motifs, minor-pentatonic questions, blues or chromatic tension, half cadences, repeated tails, syncopation, sparse long tones, dense eighth-note motion, arch contours, and large leaps followed by stepwise filling.

The 40 real calls in Call50 are extracted from POP909 [18] using a conservative two-bar melody-sampling procedure. The extraction script scans MIDI tracks with melody-like names such as `melody`, `main`, `vocal`, and `lead`, while excluding tracks that are more likely to represent

accompaniment or percussion, such as **chord**, **bass**, and **drum**. Candidate windows are two bars long, corresponding to 8 beats, and the window advances by 2 beats. A candidate is kept only if it contains between 6 and 24 notes, has a polyphonic-overlap ratio no greater than 0.10, and reaches a major/minor scale-fit score of at least 0.85. A fixed random seed, 20260529, is then used to shuffle the candidates and select 40 real snippets reproducibly.

Second, Call100 copies Call50 into a new benchmark directory and adds 50 public MIDI excerpts without rewriting the original `calls_50` directory. In the reproducible run reported here, the available public source was POP909, so all additions come from POP909. The builder constructs two-bar windows from tracks such as **MELODY**, **BRIDGE**, and **PIANO**, computes descriptive features, scores candidates against eight diagnostic target categories, and applies fingerprint deduplication.

Table 1 summarizes the source distribution, Call50 origins, and public-addition targets. The benchmark is diagnostic rather than corpus-representative: it preserves artificial stress cases while adding public excerpts for broader melodic input. Planned MAESTRO, ASAP, Lakh/Slakh, and GiantMIDI sources were not locally available, so the dataset report records their absence and the public additions are backfilled from POP909.

Table 1: Detailed composition of the Call100 benchmark

Source or origin	Subset or target type	Calls	Purpose or selection rule
<code>calls_50</code>	Artificial stress tests	10	Hand-designed edge cases: pentatonic motif, minor question, blues/chromatic tension, half cadence, repetition tail, syncopation, sparse long tones, dense motion, arch contour, and leap gap filling.
<code>calls_50</code>	Real POP909 snippets	40	Two-bar monophonic melody windows from melody-like tracks; 6–24 notes, overlap ratio ≤ 0.10 , scale fit ≥ 0.85 , seed 20260529.
POP909	Clear motif	12	Public additions selected for stable motivic structure.
POP909	Expressive rubato	8	Irregular timing and flexible inter-onset intervals.
POP909	Chord/arpeggio texture	8	Chordal or partially polyphonic input gestures.
POP909	Sparse short phrase	5	Low-information calls with few notes or long gaps.
POP909	Dense fast motion	5	High onset density and faster melodic movement.
POP909	Chromatic outside notes	5	Lower scale-fit or chromatic-tension cases.
POP909	False-ending pause	4	Internal or trailing pauses that may confuse endpoint detection.
POP909	Repetition tail	3	Repeated endings that may trigger response repetition.
Total	Call100 benchmark	100	10 artificial stress calls, 40 real Call50 snippets, and 50 public POP909 additions.

5.3 Candidate Strategies and Parameter Presets

The formal experiment compares three candidate strategies, summarized in Table 2. The raw AMT condition directly uses the pretrained AMT decoder without external phrase-level control. The controlled AMT condition keeps the same decoder but adds call profiling, prompt cleaning, repetition suppression, fallback generation, and duration matching. The motif baseline does not use a neural decoder; instead, it produces responses through rule-based motif transformation and is treated as a strong structural baseline.

Table 2: Candidate strategies in the Call100 experiment

Candidate ID	Description
<code>amt_small_raw</code>	Raw AMT response generation without external phrase-level control.
<code>amt_small_controlled</code>	AMT response generation with call profiling, repetition suppression, fallback generation, and duration matching.
<code>motif_transform_baseline</code>	Rule-based motif transformation baseline used as a strong non-neural comparison.

The experiment uses six parameter presets, listed in Table 3. Each call, preset, and candidate combination is run for 15 trials. The total experiment scale is therefore:

$$100 \text{ calls} \times 6 \text{ presets} \times 3 \text{ candidates} \times 15 \text{ trials} = 27000. \quad (28)$$

Table 3: Parameter presets used in the Call100 experiment

Preset group	Preset ID
No-theory control	<code>pentatonic_no_theory</code>
Balanced setting	<code>pentatonic_balanced</code>
Conservative setting	<code>pentatonic_conservative</code>
Creative setting	<code>pentatonic_creative</code>
Wide creative setting	<code>pentatonic_creative_wide</code>
Low-temperature setting	<code>pentatonic_low_temp_no_strongbeat</code>

Each preset/candidate pair contains 1500 samples. The harmonized audit confirms 27,000 rows, 1800 preset/candidate/call groups, 15 trials per group, no missing MIDI, and evaluator/input SHA-256 provenance. The AMT batches use experiment seed 20260622; the motif batch is re-scored, not regenerated.

5.4 Objective Metric System

The versioned evaluator `structural_compliance_v1.1` defines the closeness function

$$c(x; a, b) = \text{clip} \left(1 - \frac{|x - a|}{b}, 0, 1 \right) \quad (29)$$

and six component scores:

$$\mathcal{T} = (\text{PSR} + \text{Cad} + \text{SB})/3, \quad (30)$$

$$\mathcal{R} = (\text{QN} + \text{QRF} + \text{G})/3, \quad (31)$$

$$\mathcal{I} = 0.60 \text{SW} + 0.40(1 - \text{TSR}), \quad (32)$$

$$\mathcal{P} = \text{clip}[1 - 2 \text{CPR} - 0.2 \max(0, L_{\text{run}} - 2), 0, 1], \quad (33)$$

$$\mathcal{D} = 0.55c(H_{\text{pc}}; 2.0, 0.65) + 0.45c(U_{\text{pc}}; 5, 2), \quad (34)$$

$$\mathcal{C} = c(Z; 0.65, 0.35). \quad (35)$$

PSR, cadence (Cad), and strong-beat stability (SB) use the configured C-major-pentatonic reference; SW is the share of intervals no larger than 3 semitones and TSR the share larger than 7. QN counts durations of at least 0.125 beats; QRF accepts distances no greater than 0.065 beats from $\{0.125, 0.25, 0.375, 0.5, 0.75, 1, 1.5, 2, 3, 4\}$; G compares adjacent bars on a 16-slot binary onset grid. H_{pc} is pitch-class entropy, U_{pc} the number of used pitch classes, and Z the zlib/raw-byte ratio of rounded onset–duration–pitch tuples. Empty responses receive zero on every score.

The reported structural-compliance composite (SCC; retained as `objective_score` in legacy CSV fields) is exactly

$$\text{SCC} = 0.22\mathcal{T} + 0.20\mathcal{R} + 0.18\mathcal{I} + 0.16\mathcal{P} + 0.14\mathcal{D} + 0.10\mathcal{C}. \quad (36)$$

To expose direct optimization by A5, it is decomposed as

$$S_{\text{style}} = \frac{0.22\mathcal{T} + 0.20\mathcal{R}}{0.42}, \quad S_{\text{nonstyle}} = \frac{0.18\mathcal{I} + 0.16\mathcal{P} + 0.14\mathcal{D} + 0.10\mathcal{C}}{0.58}, \quad (37)$$

so $\text{SCC} = 0.42S_{\text{style}} + 0.58S_{\text{nonstyle}}$. The weights and normalizations are fixed code parameters, not learned from listener ratings. SCC measures rule/structure compliance, not musical quality; human ratings remain a separate outcome [7, 10].

5.5 Evaluation Integrity and Shared Batches

A reproducibility audit found that the legacy candidate runner labeled `amt_small_raw` after applying style projection, producing 0.683141; that summary is excluded. The harmonized comparison uses the exact A0 raw and A6 controlled batches from the 63,000-row ablation, paired by preset, call, trial, and seed, and retains the rule-based motif batch. All 27,000 MIDI files were re-scored by the same evaluator; maximum score drift from stored values was 5×10^{-7} and no row exceeded that tolerance. Thus Tables 4 and 7 share identical raw/controlled outputs and means; no AMT output was regenerated to repair the comparison.

5.6 Overall Candidate Comparison and Statistical Significance

Table 4 reports 9000 harmonized observations per strategy. Controlled AMT raises SCC from 0.560968 to 0.732610; the motif baseline remains higher at 0.752682.

Table 4: Overall comparison of candidate strategies

Candidate strategy	Samples	SCC	Style	Non-style
Motif baseline	9000	0.752682	0.854195	0.679172
Controlled AMT (A6)	9000	0.732610	0.809814	0.676704
Raw AMT (A0)	9000	0.560968	0.563724	0.558971

Harmonized Call100 candidate comparison

Shared A0/A6 batches; 27,000 MIDI files re-scored with structural_compliance_v1.1

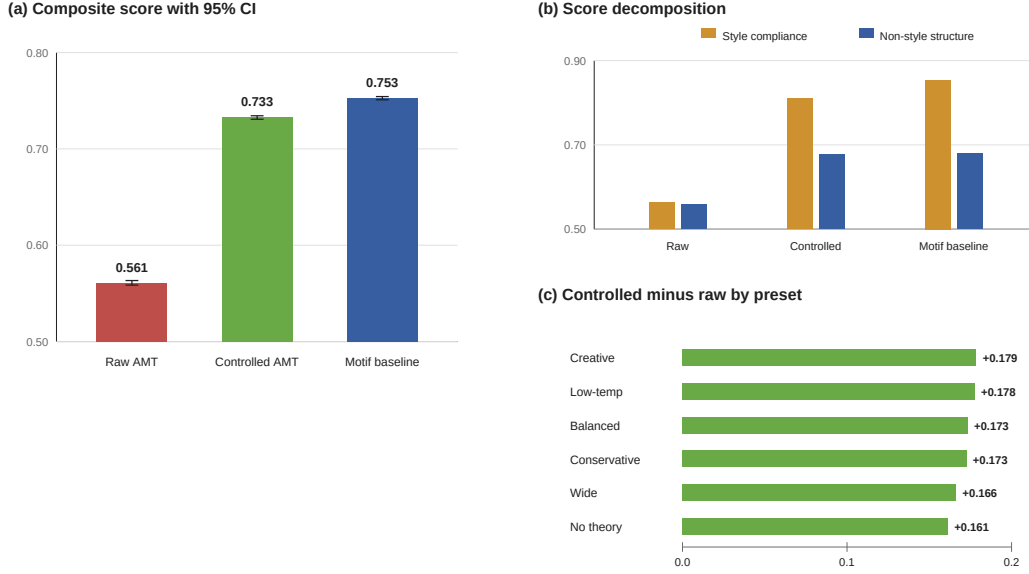


Figure 3: Harmonized Call100 comparison. Panel (a) reports SCC with 95% confidence intervals, panel (b) separates style and non-style components, and panel (c) shows controlled-minus-raw SCC by preset.

Paired differences in Table 5 use the shared preset/call/trial key. Controlled minus raw is +0.171642 (95% bootstrap CI [0.168923, 0.174287], $d_z = 1.295$), while controlled minus motif is -0.020072. Numerical underflow is reported as $p < 10^{-6}$, never $p = 0$.

Table 5: Paired candidate comparisons under the harmonized evaluator

Comparison	Mean difference	Bootstrap 95% CI	paired t p
controlled - raw	+0.171642	[0.168923, 0.174287]	$< 10^{-6}$
controlled - motif	-0.020072	[-0.022331, -0.017814]	$< 10^{-6}$
motif - raw	+0.191714	[0.188850, 0.194556]	$< 10^{-6}$

Controlled AMT exceeds raw AMT under every preset, but the motif baseline leads both score families. The claim is therefore improved structural compliance over raw AMT, not baseline dominance or listener preference.

5.7 Controlled-AMT Module Ablation

The candidate-level Call100 experiment shows that controlled AMT improves over raw AMT, but not which components drive the gain. Table 6 defines a module-level ablation using the same 100 calls, six presets, seven variants, and 15 trials for each preset/variant/call combination:

$$100 \times 6 \times 7 \times 15 = 63000. \quad (38)$$

The final validation confirms 63,000 actual rows, 4200 preset/variant/call groups, 15 trials per group, no missing SCC values, no missing response MIDI files, no unknown call IDs, and validation status **passed**.

Table 6: Controlled-AMT ablation variants

Variant	Module added	Description
A0	Raw AMT	No external control.
A1	Prompt cleaning	Only tail-repeat compression and prompt cleaning are enabled.
A2	Repetition suppression	Adds generation-time repetition suppression and resampling.
A3	Duration matching	Adds response duration and note-count matching.
A4	Motif fallback	Adds motif fallback for empty or invalid neural output.
A5	Style constraint	Adds pentatonic, two-bar, and 4/4 response style constraints.
A6	Full controlled	All controlled-AMT modules are enabled.

Table 7: Ablation summary by variant over Call100

Variant	SCC	Style compliance	Non-style structure
A0 raw AMT	0.560968	0.563724	0.558971
A1 prompt cleaning	0.574265	0.566740	0.579714
A2 repetition suppression	0.610704	0.561879	0.646061
A3 duration matching	0.640526	0.575326	0.687740
A4 fallback enabled	0.640526	0.575326	0.687740
A5 style constraint	0.708929	0.764076	0.668994
A6 full controlled	0.732610	0.809814	0.676704

Table 7 reports SCC and its two predeclared components over 9000 samples per variant. A0 to A6 gains 0.171642 in SCC and 0.117733 in non-style structure.

A0-A6 structural score decomposition

Composite = 0.42 style compliance + 0.58 non-style structure; n=9,000 per variant

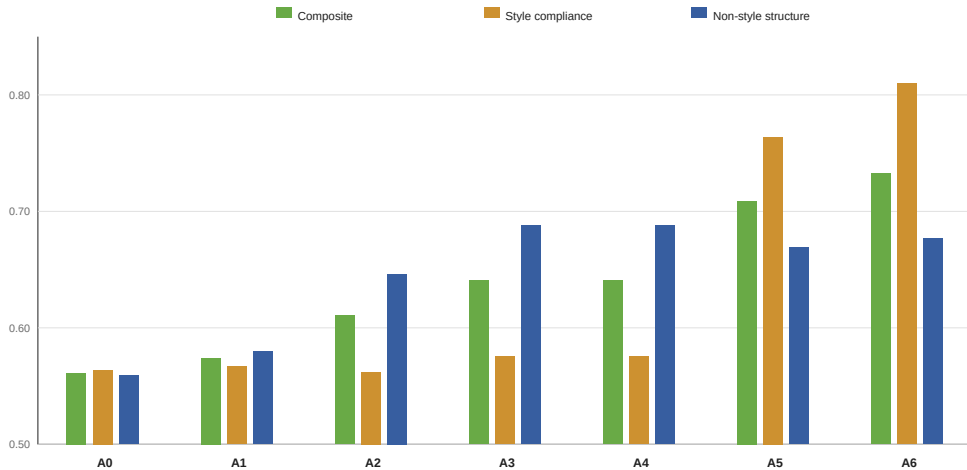


Figure 4: A0–A6 score decomposition. Grouped bars report SCC, style compliance, and non-style structure over 9000 samples per variant.

Figure 4 makes the A5 mechanism visible: style compliance rises by 0.188750, while non-style structure decreases by 0.018746. The SCC increase at A5 is therefore a constraint-compliance

manipulation check, not independent evidence of musical quality.

Table 8: Stepwise module contributions to SCC

Step	Module added	Mean delta	95% CI	p
A1-A0	Prompt cleaning	0.013297	[0.010832, 0.015736]	$< 10^{-6}$
A2-A1	Repetition suppression	0.036439	[0.034068, 0.038919]	$< 10^{-6}$
A3-A2	Duration matching	0.029822	[0.028143, 0.031398]	$< 10^{-6}$
A4-A3	Motif fallback	0.000000	[0.000000, 0.000000]	n/a (all ties)
A5-A4	Style constraint	0.068403	[0.066357, 0.070537]	$< 10^{-6}$
A6-A5	Full controlled	0.023681	[0.022480, 0.024868]	$< 10^{-6}$

Stepwise module contribution by score family

A5 directly targets style metrics; its non-style structural component decreases

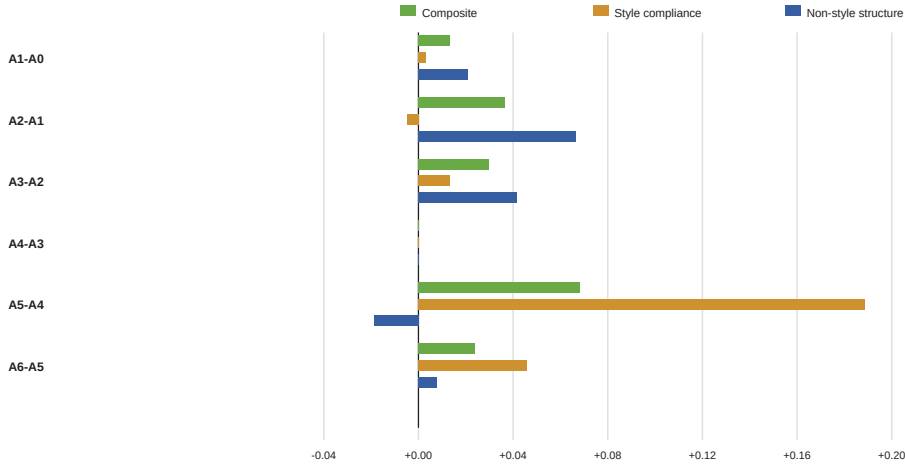


Figure 5: Stepwise changes in SCC, style compliance, and non-style structure. A5 directly optimizes the style family and slightly decreases the non-style family.

The fallback audit in Table 9 resolves the A3–A4 all-zero result. `event repair` counts individual rejected-token substitutions; `motif used` counts whole-phrase replacement.

Table 9: Fallback activation audit over 9000 samples per variant

Variant	Repair samples	Repair events	Empty before motif	Motif used
A0	0	0	0	0
A1	0	0	0	0
A2	4236	6888	0	0
A3	3712	5263	0	0
A4	3712	5263	0	0
A5	3712	5263	0	0
A6	3712	5263	0	0

The evaluator assigns SCC=0 to an empty response. No A3 response was empty, so A4 had no eligible phrase and activated motif fallback zero times; if activated, the replacement MIDI would be evaluated after generation like every other output. All 9000 A3–A4 SCC differences are exactly zero and all 9000 response MIDI files are byte-identical. Within either variant, the 5288 samples without event repair average 0.643951, while the 3712 repaired samples average

0.635647. Therefore A4 is an unexercised safety branch in this run, not evidence that fallback is effective or ineffective.

5.8 Direct MIDI-VAD Endpoint Benchmark

The endpoint detector is evaluated directly by replaying all 100 Call100 files at 5 ms polling resolution. The final Note-Off of each isolated file is a reproducible proxy boundary. For each post-boundary deadline $d \in \{0.5, 1.0, 2.0\}$ s, at most one commit in $[T_{\text{ref}} - 0.1, T_{\text{ref}} + d]$ is a true positive; other commits are false positives and an unmatched file is a false negative. The two-second deadline is the primary table, while all deadlines remain in the archive. This proxy does not replace human phrase-boundary annotation.

The main conditions compare adaptive MIDI-VAD with clustering/confirmation ablations and fixed 300, 500, and 800 ms silence thresholds. All fixed baselines retain the 80 ms cluster and 150 ms confirmation settings. Table 10 reports 100 calls per condition; F1 intervals use 2000 call-level bootstrap resamples.

Table 10: Call100 endpoint results at the two-second post-boundary deadline

Condition	Precision	Recall	F1	Premature	Cancel rate	Median error s
Adaptive full	0.624	0.830	0.712	38	0.070	0.796
No clustering	0.610	0.830	0.703	42	0.087	0.796
No confirmation	0.597	0.860	0.705	49	0.000	0.632
Neither	0.571	0.840	0.680	53	0.000	0.632
Fixed 300 ms	0.250	0.760	0.376	228	0.374	0.330
Fixed 500 ms	0.427	0.880	0.575	118	0.116	0.500
Fixed 800 ms	0.575	0.920	0.708	68	0.130	0.793

Call100 MIDI-VAD endpoint benchmark

Final Note-Off proxy boundary; 100 ms early tolerance and 2 s post-boundary deadline

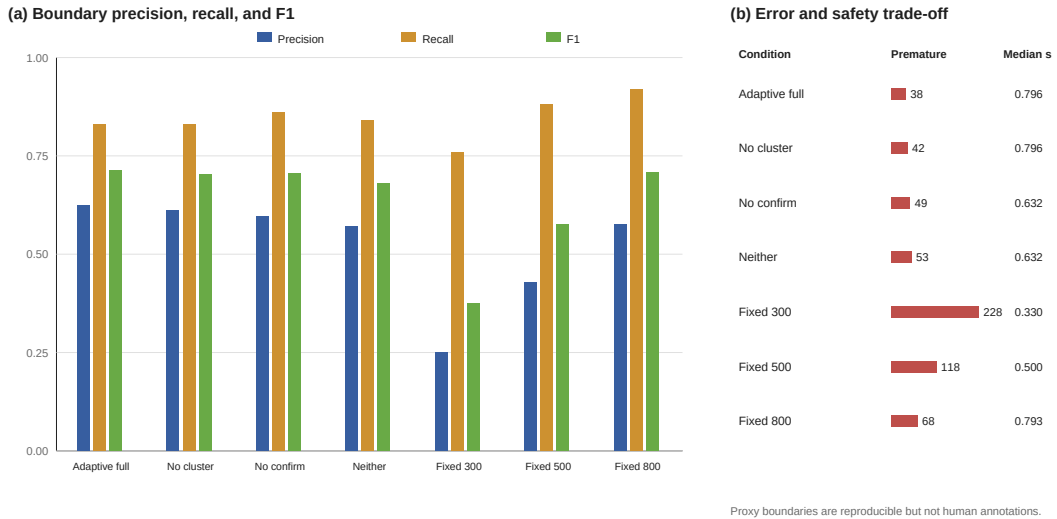


Figure 6: Direct endpoint benchmark. Panel (a) reports precision, recall, and F1; panel (b) shows premature commits and median signed boundary error.

Adaptive full yields $F1=0.712$ (95% CI $[0.638, 0.784]$), comparable to fixed 800 ms at 0.708

[0.646, 0.767], while reducing premature commits from 68 to 38; the overlapping intervals do not establish superiority. Its recall is 0.14, 0.58, and 0.83 at the 0.5, 1.0, and 2.0 s deadlines, respectively, exposing the latency cost of conservative segmentation. Cluster-window sensitivity gives F1 values 0.703, 0.715, 0.712, and 0.704 at 0, 40, 80, and 120 ms. Confirmation sensitivity gives 0.705, 0.695, 0.712, and 0.728 at 0, 75, 150, and 250 ms, while median error rises from 0.632 to 0.705, 0.796, and 0.896 s. Thus 80/150 ms is a middle operating point, not a uniquely optimal constant.

5.9 Runtime Latency Logging and Scheduler Replay

SCC does not characterize scheduler responsiveness. The latency experiment therefore replays actual local AMT decoding timings collected during the A6 runs. The machine is a Lenovo 82JQ running 64-bit Windows 11 Pro, with an AMD Ryzen 7 5800H (8 cores/16 logical processors), 15.86 GiB physical RAM, and an NVIDIA GeForce RTX 3060 Laptop GPU with 6144 MiB VRAM. L0 starts decoding at endpoint commit. L1 overlaps decoding only with the confirmation interval, so $\Delta T_{\text{preload}} = \Delta T_{\text{confirm}} = 150$ ms; no additional 250 ms horizon exists. Both use an 80 ms micro-buffer and underrun deadline. Inference timings are observed, whereas endpoint-to-first-MIDI values are deterministic scheduler replay rather than direct live-performance measurements.

The primary latency metric is endpoint-to-first-MIDI latency:

$$L_{\text{first}} = t_{\text{first_midi_out}} - t_{\text{endpoint_commit}}. \quad (39)$$

This metric measures the delay between the confirmed phrase endpoint and the first scheduled MIDI response event.

Table 11: Runtime latency comparison between preload-off and preload-on conditions

Condition	Samples	Mean ms	P50 ms	P95 ms	P99 ms	Underrun
L0 preload off	9000	161.303	136.455	323.521	479.574	20.744%
L1 preload on	9000	91.721	80.000	173.521	329.574	5.244%

Table 12: Paired latency reduction from speculative preload

Comparison	Paired samples	Mean reduction ms	95% CI
L1 preload on vs. L0 preload off	9000	69.582	[68.865, 70.263]

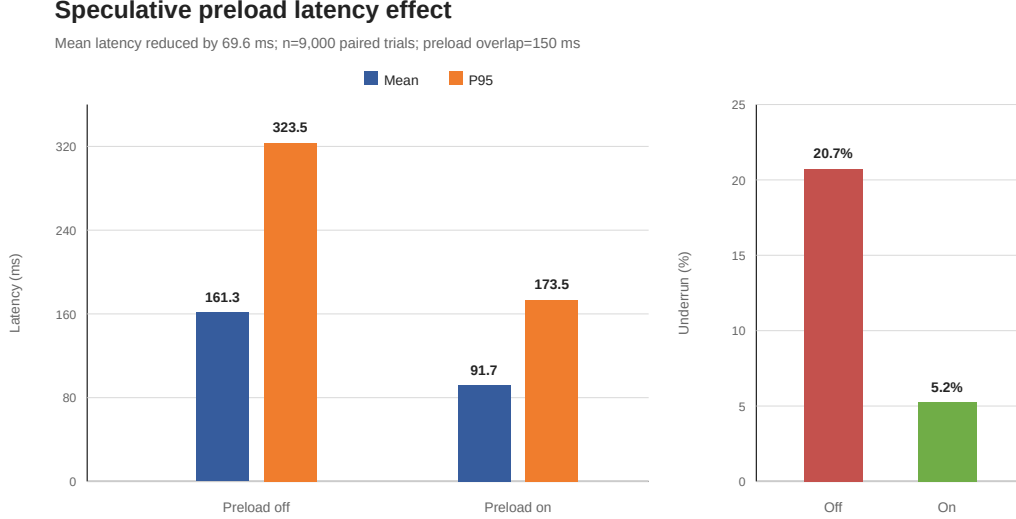


Figure 7: Scheduler-replay comparison with preload off and with 150 ms of pre-commit overlap. The left panel reports mean and P95 endpoint-to-first-MIDI latency; the right panel reports buffer-underrun rate.

Tables 11 and 12, together with Figure 7, show the replay results. All 9000 paired differences favor preload; an exact two-sided paired sign test gives $p < 10^{-6}$. Preload reduces mean endpoint-to-first-MIDI latency from 161.303 ms to 91.721 ms, reduces P95 from 323.521 ms to 173.521 ms, and lowers buffer-underrun rate from 20.744% to 5.244%. This improves scheduled post-endpoint responsiveness by moving up to 150 ms of decoding into the candidate-confirmation interval; it does not reduce intrinsic AMT decoding time.

5.10 Dataset Source Robustness and Failure-Case Analysis

Table 13 applies the harmonized evaluator to source groups. The original Call50 subset remains easier than the added POP909 subset.

Table 13: Performance by source dataset and origin

Grouping field	Value	Samples	Mean SCC
source dataset	<code>calls_50</code>	13500	0.702215
source dataset	<code>POP909</code>	13500	0.661958
origin	<code>artificial_stress</code>	2700	0.725250
origin	<code>real_public_dataset</code>	10800	0.696456
origin	<code>public_midi_extract</code>	13500	0.661958

This difference does not indicate failure on POP909; it shows that the added public excerpts are harder. The POP909 subset has a higher average maximum interval, 12.093 versus 8.360 for `calls_50`, and a higher average note count, 15.905 versus 13.588, increasing the difficulty of response planning and duration matching.

Category extremes in Table 14 similarly identify sparse and rubato inputs as difficult under SCC.

For A6, event repair occurs in 3712/9000 samples (41.24%; 5263 repaired events), but whole-phrase motif fallback occurs zero times. Reporting the former as a “fallback rate”

Table 14: Examples of high- and low-scoring categories

Category	Samples	Mean SCC	Interpretation
<code>pentatonic_clear_motif</code>	270	0.775829	Clear motif and stable scale structure.
<code>modal_question</code>	270	0.774319	Clear question-like phrasing and cadential target.
<code>syncopation</code>	270	0.753149	Distinct rhythmic profile remains structurally manageable.
<code>expressive_rubato</code>	2160	0.632543	Free timing increases rhythm and duration-matching difficulty.
<code>sparse_short</code>	1350	0.620874	Low input information makes structural filling less stable.
<code>sparse_cadential</code>	270	0.618459	Sparse cadential fragments often lead to short or weak responses.

caused the earlier 41.46% ambiguity; the public schema now stores `event_repair_count` and `motif_fallback_used` separately.

5.11 Anonymous Blind Listening Study

Stimuli and procedure. The subjective experiment used a single-stimulus, fully randomized design. Each participant rated eight anonymous piano-rendered clips constructed from four call contexts: two POP909 excerpts and two artificial stress cases. Within each context, both conditions used the same call and only the response differed. Two contexts compared raw AMT (A0) with controlled AMT (A4), while two compared the motif-transformation baseline with full controlled AMT (A6). A4 cumulatively activates prompt cleaning, repetition suppression, duration matching, and a fallback-capable branch; A6 additionally activates the style and full-theory controls. All responses used the `pentatonic_conservative` preset. For each call-condition combination, the displayed response was the trial whose SCC was closest to the median of 15 trials. Every selected response had `event_repair_count`=0 and `motif_fallback_used`=0, so no listening stimulus contained event repair or whole-phrase replacement.

Each participant evaluated all eight clips in an independently randomized order. The interface disclosed neither model identity nor condition label and showed no MIDI visualization. For each item, listeners could play synchronized renderings of the full call plus response, the call alone, and the response alone. They then answered four mandatory questions about (1) transition fluency, (2) phrase development and closure, (3) rhythmic and musical correspondence with the call, and (4) overall acceptability. Each question used a forced-choice four-point scale from 1 to 4, with no neutral option. Playback counts, presentation position, per-item dwell time, and total study duration were recorded.

Participants and quality control. Recruitment was anonymous and conducted through WeChat using the mainland-China deployment. Participants acknowledged that their anonymous ratings would be used for academic analysis before starting, and no direct personal identifiers were requested. The frozen export contained 44 complete submissions and 1,408 rating rows, exactly 32 rows per submission. Before unblinding, two submissions were excluded because at least one item had no recorded playback, and the later of two submissions sharing one session identifier was removed. This left 41 participants and 1,312 ratings for the primary analysis. One participant used the same score throughout; this response was retained under the

locked rule, and excluding it in sensitivity analysis did not change either conclusion. Among included participants, 15 reported 0–2 years, 17 reported 3–5 years, and 9 reported more than 5 years of musical experience; 25 reported high and 16 low familiarity with improvisation. Listening devices were headphones ($n = 23$), phones ($n = 13$), and speakers ($n = 5$). Median completion time was 322 seconds.

Analysis. For each participant and comparison family, ratings were averaged over the four questions and two matched call contexts. Primary effects are paired participant-level mean differences, defined as controlled minus comparator. Confidence intervals were obtained with 50,000 participant-bootstrap draws, and two-sided p values with 200,000 sign-flip permutations. Cohen’s d_z summarizes paired effect size. A secondary linear model adjusted for presentation index and call-pair fixed effects while including participant fixed effects and participant-clustered standard errors. The four questions showed high internal consistency across participant-item observations (Cronbach’s $\alpha = 0.939$).

Table 15: Blind-listening results on a four-point scale ($N = 41$). Differences are controlled minus comparator; confidence intervals use participant bootstrap resampling.

Comparison	Comparator	Controlled	Difference (95% CI)	p_{perm}
Controlled A4 – raw AMT	2.945	2.787	−0.159 [−0.326, 0.006]	0.0785
Controlled A6 – motif baseline	2.256	1.909	−0.348 [−0.534, −0.162]	0.0009

Results. Table 15 does not support a perceptual advantage for controlled AMT on the selected stimuli. In the A4 comparison, controlled AMT was rated 0.159 points below raw AMT, but the interval included zero ($d_z = -0.289$, $p = 0.0785$). After adjustment for presentation position and matched call pair, the estimated difference remained similar at -0.164 (95% CI [−0.352, 0.025], approximate $p = 0.090$). The correct interpretation is uncertainty rather than equivalence or superiority.

In the A6 comparison, controlled AMT was rated 0.348 points below the motif baseline (95% CI [−0.534, −0.162], $d_z = -0.570$, $p = 0.0009$). The adjusted estimate was -0.349 (95% CI [−0.567, −0.131], approximate $p = 0.0017$), preserving the conclusion after accounting for order and call pair. Ratings increased by approximately 0.038 points per later presentation position (95% CI [0.009, 0.068]), which justifies retaining order in the adjusted model but does not explain the condition contrast. Excluding the one straight-line participant yielded nearly identical differences (-0.163 and -0.356).

Across the eight fixed stimuli, SCC and mean human rating showed little descriptive alignment (Pearson $r = -0.054$; Spearman $\rho = 0.071$). With only eight selected stimuli this is exploratory. Structural compliance improves over raw AMT at scale, but these exemplars do not show corresponding listener preference; the motif baseline is preferred to A6 on this restricted test.

5.12 Interpretation and Scope of Evaluation

The endpoint benchmark uses file-end proxies rather than human boundaries, motif fallback never activates in A0–A6, and scheduler replay does not measure faster AMT inference. The blind study contains 41 analyzed participants from one Chinese social channel, four contexts, one preset, and eight fixed, repair-free stimuli. No live co-performance user study is included, so perceived turn taking and broader musical usefulness remain unverified.

6 Conclusion

The primary contribution is an external method that places adaptive turn detection, phrase control, and speculative scheduling around a frozen offline AMT. Direct endpoint replay shows a precision–latency trade-off rather than universal segmentation accuracy. Shared-batch evaluation raises SCC from 0.560968 to 0.732610 and non-style structure from 0.558971 to 0.676704; A5’s larger style gain is explicitly separated from that evidence. A 150 ms overlap lowers replayed scheduler latency from 161.303 to 91.721 ms, but the listening study does not establish perceptual superiority. Future work should use human-labeled phrase boundaries, live interaction, and broader pre-registered listening samples.

Data and Code Availability

The research code, paper source, trial-level structural metrics, endpoint and latency summaries, aggregate listening results, and release documentation are available in the public GitHub repository. The reproducibility archive corresponding to this manuscript is GitHub release v1.1.0. Zenodo archives the release under the stable all-versions concept DOI; the release page links its exact version DOI. A browsable project page is also available.

The public archive includes the implementation, exact score specification, path-redacted trial metrics, source hashes, statistical summaries, and tests needed to inspect the reported structural, endpoint, scheduling, and aggregate listening results. Participant-level response exports, exclusion identifiers, private answer keys, and deployment credentials are retained privately. Large or license-sensitive runtime artifacts, including pretrained model weights, third-party audio tools, sample libraries, source datasets, and generated per-run MIDI batches, are not redistributed and should be obtained separately under their respective licenses.

AI Tool Usage Statement

AI tools were used in this project as auxiliary research, writing, and programming support. This disclosure refers to general-purpose AI assistance used during project preparation; the autoregressive Transformer system evaluated in this paper is part of the research object itself and is described separately in the methodology and experiments.

The authors used OpenAI ChatGPT/Codex in three main places. First, during software development, AI assistance was used to inspect Python and MIDI-processing code, identify possible implementation errors, debug LaTeX or Python issues, and draft small validation or analysis procedures. All code execution, experiment running, and result validation were performed and checked by the authors. Second, during manuscript preparation, AI assistance was used to improve English wording, reorganize paragraphs, refine LaTeX formatting, and make tables and figure descriptions clearer. Third, during result reporting, AI assistance was used to help summarize verified CSV and log outputs and convert checked numerical results into concise academic prose.

AI tools were not used to fabricate experimental data, replace the Call100 or blind-listening evaluations, make unsupervised final research decisions, or generate references accepted without checking. All numerical claims, tables, figures, citations, and final conclusions were reviewed by the authors against the frozen experiment outputs, source code, and cited literature.

References

- [1] Nicolas Boulanger-Lewandowski, Yoshua Bengio, and Pascal Vincent. Modeling temporal dependencies in high-dimensional sequences: Application to polyphonic music generation

- and transcription. In *Proceedings of the 29th International Conference on Machine Learning*, pages 1159–1166, 2012.
- [2] Daryl J. Daley and David Vere-Jones. *An Introduction to the Theory of Point Processes: Volume I: Elementary Theory and Methods*. Springer, New York, 2 edition, 2003.
 - [3] Roger B. Dannenberg. An on-line algorithm for real-time accompaniment. In *Proceedings of the 1984 International Computer Music Conference*, pages 193–198, 1985. URL <https://www.cs.cmu.edu/~rbd/papers/icmc84accomp.pdf>.
 - [4] Douglas Eck and Jürgen Schmidhuber. Finding a temporal structure in music with lossy lstm recurrent networks. *Neural Computation*, 14(11):2607–2625, 2002.
 - [5] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020.
 - [6] Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Noam Shazeer, Ian Simon, Curtis Hawthorne, Andrew M. Dai, Matthew D. Hoffman, Monica Dinculescu, and Douglas Eck. Music transformer: Generating music with long-term structure. In *International Conference on Learning Representations*, 2019.
 - [7] Shulei Ji, Xinyu Yang, and Jing Luo. A survey on deep learning for symbolic music generation: Representations, algorithms, evaluations, and challenges. *ACM Computing Surveys*, 56(1):1–39, 2023.
 - [8] Tornike Karchkhadze and Shlomo Dubnov. Towards real-time human-ai musical co-performance: Accompaniment generation with latent diffusion models and max/msp. arXiv preprint arXiv:2604.07612, 2026.
 - [9] Nelson Posse Lago and Fabio Kon. The quest for low latency. In *Proceedings of the International Computer Music Conference*, pages 33–36, 2004. URL <https://www.ime.usp.br/~kon/papers/icmc04-latency.pdf>.
 - [10] Alexander Lerch, Claire Arthur, Nick Bryan-Kinns, Corey Ford, Qianyi Sun, and Ashvala Vinay. Survey on the evaluation of generative models in music. *ACM Computing Surveys*, 2025.
 - [11] Søren Tjagvad Madsen and Gerhard Widmer. Separating voices in midi. In *Proceedings of the 7th International Conference on Music Information Retrieval*, pages 57–60, 2006. URL <https://archives.ismir.net/ismir2006/paper/000163.pdf>.
 - [12] Mark Nelson and Belinda Thom. A survey of real-time midi performance. In *Proceedings of the International Conference on New Interfaces for Musical Expression*, pages 35–38, 2004. doi: 10.5281/zenodo.1176643.
 - [13] Sageev Oore, Ian Simon, Sander Dieleman, Douglas Eck, and Karen Simonyan. This time with feeling: Learning expressive musical performance. *Neural Computing and Applications*, 32:955–967, 2020.
 - [14] Nicola Privato, Omar Rampado, and Alberto Novello. Scramble live: Combining lstm and markov chains for real-time musical interaction. In *Proceedings of the 19th Sound and Music Computing Conference*, 2022. doi: 10.5281/zenodo.6800786.
 - [15] Cristina Rottondi, Chris Chafe, Claudio Allocchio, and Augusto Sarti. An overview on networked music performance technologies. *IEEE Access*, 4:8823–8843, 2016. doi: 10.1109/ACCESS.2016.2628440.

- [16] John Thickstun, David Hall, Chris Donahue, and Percy Liang. Anticipatory music transformer. *arXiv preprint arXiv:2306.08620*, 2023.
- [17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [18] Ziyu Wang, Ke Chen, Junyan Jiang, Yiyi Zhang, Maoran Xu, Shuqi Dai, Xianbin Gu, and Gus G. Xia. Pop909: A pop-song dataset for music arrangement generation. In *Proceedings of the 21st International Society for Music Information Retrieval Conference*, pages 38–45, 2020.